

Hospital Management System — Exam Demo Documentation (Auth, RBAC, OOP, SHA-256)

0. What you're presenting

This documentation explains, at a **presentation/exam-friendly level**, what the system implements for:

- **Authentication & sign-in / sign-up**
 - **OTP verification**
 - **Password hashing**
 - **SHA-256 usage**
 - **RBAC (Role-Based Access Control)**
 - **Object-Oriented Programming (OOP) concepts & design principles**
 - **Key features and the specific changes made during this work**
-

1) Project architecture (Separation of concerns)

Your codebase follows a practical separation between:

A. UI layer (FXML + Controllers)

- **FXML files** define screens (layout/components).
- **Controllers** handle UI events and connect UI to services.

Examples:

- `LoginController` — sign in UI logic
- `SignUpController` — sign up UI logic
- `OTPVerificationController` — OTP screen UI logic
- `DashboardController` — dashboard UI logic (charts + appointment actions)
- `HealthRecordController` — health records UI logic

B. Business logic layer (Services)

Services contain the “rules” of the system:

- `AuthService` — login/register/password change logic
- `OTPService` — generate OTP, hash OTP, store/verify OTP, activate accounts
- `AppointmentService` — booking/confirm/cancel/complete/reschedule + scheduling validation

C. Data access layer (DAOs)

DAOs perform DB interactions:

- `UserDAO`, `DoctorDAO`, `PatientDAO`
- `AppointmentDAO`
- `HealthRecordDAO`

- **OTPDao** (for OTP persistence)

D. Domain models (Entities)

Models represent data objects used across layers:

- **User, Doctor, Patient**
- **Appointment**
- **HealthRecord**

Exam point: This separation demonstrates **SRP (Single Responsibility Principle)** and makes code easier to test and maintain.

2) OOP concepts & design principles used

2.1 Encapsulation

Key data is hidden inside classes, exposed through methods.

- **AuthService** hides hashing and verification details.
- **OTPService** hides OTP hashing + expiry rules.
- DAOs hide SQL and mapping to models.

2.2 Abstraction

Complex processes are presented with simple methods:

- **AuthService.login(email, password)**
- **OTPService.requestOtp(userId, destination)**
- **OTPService.verifyOtpAndActivate(userId, otp)**
- **AppointmentService.confirmAppointment(...), cancelAppointment(...), rescheduleAppointment(...)**

2.3 Interface/Polymorphism (Notifier)

OTPService depends on a **Notifier** abstraction:

- Console notifier used for OTP sending This enables swapping output channels without changing OTP logic.

2.4 Singleton pattern (SessionManager)

SessionManager is used to store the **current logged-in user** globally:

- **SessionManager.setCurrentUser(user)**
- **SessionManager.getCurrentUser()**

2.5 Composition

Services compose DAOs and other helpers:

- Example: `OTPService` uses `OTPDOA0`, `UserDAO`, and `Notifier`.
-

3) Authentication: Sign up, OTP verification, Sign in

3.1 Sign Up flow (Controller → Service → DAO)

Main entry: `SignUpController.onSignUp(...)`

High level steps:

1. User selects role: **PATIENT** or **DOCTOR**
2. Input validation (role-specific required fields)
3. Background task:
 - Build the corresponding entity (`Patient` or `Doctor`)
 - Call `AuthService.register(entity, password)`
 - Insert role-specific row(s) (patients/doctors)
 - For doctors: create working schedule rows
4. Request OTP:
 - `OTPService.requestOtp(userId, email)`
5. Store pending signup state:
 - `PendingSignupSession.setPendingUserId(...)`
 - `PendingSignupSession.setPendingDestination(...)`
6. Navigate to OTP screen:
 - `OTPVerification.fxml`

3.2 OTP verification (Activating the account)

Main entry: `OTPVerificationController.onVerify(...)`

High level steps:

1. Validate there is a pending signup session
2. Read OTP entered by user
3. Call:
 - `OTPService.verifyOtpAndActivate(userId, otp)`
4. If valid:
 - mark user as verified (`UserDAO.markVerified`)
 - clear pending session
 - return to Login screen

3.3 Sign In flow

Main entry: `LoginController.onLogin(...)`

High level steps:

1. Validate email format + password non-empty
2. Call:
 - `AuthService.login(email, password)`

3. In `AuthService.login(...)`:

- fetch user by email (`UserDAO.findByEmail`)
 - verify password using BCrypt
 - enforce OTP verification:
 - if `!user.isVerified()` login fails
-

4) Password hashing vs SHA-256 (Important exam clarification)

4.1 Passwords: BCrypt (NOT SHA-256)

Passwords are hashed with **BCrypt**:

In `AuthService.java`:

- Register:
 - `BCrypt.hashpw(rawPassword, BCrypt.gensalt(12))`
- Login:
 - `BCrypt.checkpw(password, user.getPasswordHash())`

✅ So in your system:

- **Passwords** → BCrypt
- **OTP** → SHA-256

4.2 OTP: SHA-256 is implemented

In `OTPService.java`, method `sha256Hex(String input)`:

- Uses:
 - `MessageDigest.getInstance("SHA-256")`
- Hashes OTP deterministically:
 - digest bytes to hex string

Then:

- Store hashed OTP + expiry in DB via `OTPDao`
- Verify user-entered OTP by hashing it again and comparing hashes

✅ Therefore SHA-256 is **implemented** and used for OTP hashing.

5) RBAC (Role-Based Access Control)

RBAC is enforced across multiple layers:

5.1 UI visibility

Controllers hide or disable buttons depending on role. Example:

- `HealthRecordController` hides add/save actions for patients.

5.2 Service/DAO authorization checks

Services/DAOs enforce authorization:

- Appointment confirmation/cancel/complete:
 - Doctors can only act on their own appointments
- Health record access:
 - Doctors can only view records for patients they have handled (via appointment linkage)
 - Patients can only view their own records
 - Admin can access broader data

Exam point: This is layered security — UI is convenience; **services/DAOs are the real enforcement.**

6) Appointment workflow (confirm/cancel/complete/reschedule)

6.1 Scheduling rules (Phase 12 behavior)

`AppointmentService` enforces:

- appointment must be at least **30 minutes in the future**
- appointment must align to working-hours schedule
- conflict detection based on 1-hour slot overlaps

6.2 Rescheduling

We implemented doctor rescheduling rules:

- update appointment datetime
 - keep status unchanged
 - enforce scheduling validation and conflicts excluding the appointment being moved
-

7) What was changed/fixed during this work (demo-ready summary)

7.1 Health Records page "Load Records does nothing"

Fixed by making the doctor's patient dropdown load only patients they have appointments with, and by adding failure handling so errors aren't silent.

Files involved:

- `HealthRecordController.java`
- `HealthRecordDAO.java` / `AppointmentDAO.java` (RBAC check alignment)

7.2 Dashboard doctor actions: popup instead of navigation

Fixed by changing dashboard appointment click behavior:

- Doctors now get a popup with **Confirm / Cancel / Reschedule**
- Clicking an appointment no longer takes the doctor to the appointment table

Files involved:

- `DashboardController.java`

7.3 Reschedule support

Added reschedule business logic and backing DAO operations:

- `AppointmentService.rescheduleAppointment(...)`
 - `AppointmentDAO` datetime update + conflict check excluding current appointment
-

8) SHA-256 algorithm: what is implemented and what isn't

Implemented

- **OTP hashing uses SHA-256**
 - `MessageDigest.getInstance("SHA-256")`
 - convert digest to lowercase hex string

Not implemented (as SHA-256 usage)

- Password hashing does **not** use SHA-256.
 - Password hashing uses **BCrypt**.
-

9) Suggested exam "Q&A script" (short answers)

Q1: "Why did you use BCrypt?"

A: BCrypt is designed for password storage; it's slow and resistant to brute-force compared to fast hashes.

Q2: "Where exactly is SHA-256 used?"

A: In `OTPService.sha256Hex()`, OTPs are hashed using `MessageDigest.getInstance("SHA-256")` before storing/verifying.

Q3: "How do you prevent unauthorized access?"

A: RBAC checks exist in services/DAOs; controllers only change UI visibility. Services enforce the real rules.

Q4: "How do you confirm/cancel appointments safely?"

A: `AppointmentService` checks valid status transitions and enforces scheduling constraints.

10) Key files to mention during presentation

- `AuthService.java` — BCrypt password hashing + verified check
- `OTPService.java` — OTP generation + SHA-256 hashing + OTP activation
- `LoginController.java` — sign in UI flow
- `SignUpController.java` — sign up UI flow + OTP request

- `OTPVerificationController.java` — OTP verification UI flow
 - `AppointmentService.java` — confirm/cancel/complete/reschedule rules
 - `AppointmentDAO.java` — scheduling conflict checks + DB updates
 - `HealthRecordDAO.java` / `HealthRecordController.java` — RBAC for record viewing
 - `DashboardController.java` — doctor decision popup + click behavior fix
-

End

This system demonstrates clean separation of UI/controller/service/DAO/model layers, proper OOP design principles, and secured authentication with BCrypt passwords + SHA-256 OTP hashing, including role-based access control for sensitive operations.